

Item-Level Field Extraction: Deterministic Rules, Claude, and How to Tell Which Is Right

Dan Post

September 2026

Abstract

The item-level dataset has 29 fields and every one has been produced by a regular expression. This memo asks whether that is the right instrument, measures the alternative on hand-labelled data, and reports which fields need a language model, which do not, and how to run one.

On a **frozen test half of 139 hand-labelled items that no prompt was tuned against**, the deterministic extractor scores **75.2%** and Claude Haiku 4.5 with six retrieved worked examples scores **96.6%**. The gold set now spans both document eras: the model scores **97.2%** on the 1998–2014 HTML minutes and **94.0%** on the 2015–2026 PDF minutes, which is the first measurement of the modern era and the one this project was waiting on. A full corpus pass costs **\$70**.

The recommendation is Haiku 4.5 with few-shot examples, with the instructions placed ahead of the examples so the prompt prefix is cacheable. That ordering is worth \$38 across the corpus and costs nothing in accuracy — the two orderings score identically on the test half. Model choice was never the interesting decision: three models spanned three points in the first round while the gap to the regex was twenty-one, and prompting moved Haiku further than upgrading to Opus did.

Getting a trustworthy number required fixing the ruler twice. Exact string matching — the project’s original metric — charged the models about nine points for a trailing full stop and a cross-street gloss on an address; a field-aware comparison (§2.7) fixed that. The second correction was to the gold set itself: every disagreement between model and label was judged against the source block, and **52% of them were label errors, not model errors** (§4.3). An accuracy figure computed before that work is a lower bound of unknown size.

Scope. The gold set is 294 items — 233 from 1998–2014 and 61 from 2015–2026, five per year — at gold version g3. The modern stratum is thin for 30% of the corpus and is the first thing to extend.

Contents

1	The question	3
2	Part I — Methodology	3
2.1	The gold set and the frozen split	3
2.2	The methods	3
2.3	Era-matched retrieval	4
2.4	The prompt	4
2.5	What the model is told about <code>request_type</code>	4

2.6	The response schema	5
2.7	The metric: why not exact match	5
2.8	Two rates, never averaged	6
3	Part II — Results	6
3.1	The headline	6
3.2	Where the remaining error is	9
3.3	Where the regex is still the right tool	10
3.4	Where a model is not optional	10
4	Part III — How each method fails	10
4.1	Failure classes	10
4.2	<code>request_type</code> : what the prompt is doing	11
4.3	Some “model errors” are gold errors	11
4.4	The bugs the adjudication round exposed	12
4.5	Over-extraction, read carefully	12
5	Part IV — Improving the model	13
5.1	The blank policy	13
5.2	Few-shot examples	13
5.3	What is not worth doing	14
5.4	Speaker stance: what the model is asked to decide	14
6	Part V — Recommendation	14
6.1	What would change this recommendation	14
6.2	Other things worth doing, in order	15
7	Limitations	15
8	Reproducing this	16

1 The question

The pipeline’s standing rule is “deterministic rules first”. It earned that place on the date and meeting-level stages, which produce 100% item accuracy over 1,621 items and reproduce byte-identically on every run. This memo asks whether the rule also holds for the *item fields* — the 29 columns recording what was requested and what the Commission did about it.

It is not an abstract question. Those fields are the dataset. If a regular expression finds the roll call on 80% of items and a model finds it on 100%, a fifth of the Commission’s voting record is present or absent from every downstream analysis.

2 Part I — Methodology

2.1 The gold set and the frozen split

The gold set is **294 hand-labelled records** at gold version **g3**: 233 from the 1998–2014 HTML minutes and 61 from the 2015–2026 PDF minutes, the latter drawn five per year so that every year of the modern era is represented.

Those 294 are split **155 train / 139 test**, stratified by era and then by year, with a fixed seed and a SHA-256 over the gold records so drift is detectable (`gold_split.py --verify`). Stratifying by year matters because the archive’s layout drifts; a split loading one side with 1998 pages would be easier or harder for a reason unrelated to method.

The modern items were added by `gold_split.py --extend`, which assigns only the ids that were not previously on a side. This is not a nicety. A rebuild reshuffles items that have already been looked at while tuning, which silently moves test items into train and destroys the only thing the split is for.

Gold is versioned, and every number names its version. Working the adjudication queue changes gold, which changes every frozen test number, so a snapshot is registered under a name (**g1**, **g2**, **g3**) with its content hash before any round of scoring. The numbers in this memo are all **g3**. Numbers from earlier drafts of this memo were measured against a gold set that has since been corrected in about 150 places and are not comparable.

Every headline number here is measured on the test half. Prompt variants were developed against train. This discipline exists because the moment a prompt is chosen by looking at items, those items stop measuring anything.

Sampling below the full set was considered and rejected: the whole gold set costs about \$2 per configuration, while a 25-item draw would cut per-field n to about 12 on a field the gold fills half the time — a confidence interval near ± 25 points, which cannot separate any two configurations.

2.2 The methods

The first round of this memo compared four methods — the deterministic extractor and three Claude models — and found that model choice was not the interesting decision: Haiku 4.5, Sonnet 5 and Opus 5 spanned three points while the gap to the regex was twenty-one, and adding worked examples moved Haiku further than upgrading to Opus did. That round is summarised in §5.2; this round takes Haiku 4.5 with six examples as settled and asks two remaining questions — how it does on the modern era, and what it costs to run.

Method	What it is	Configuration
<code>regex</code>	<code>autoextract.extract()</code>	the current production extractor
<code>haiku-4.5-g3</code>	<code>claude-haiku-4-5</code>	Batch API, structured outputs, 6 examples
<code>haiku-4.5-g3cache</code>	<code>claude-haiku-4-5</code>	as above, instructions first (cacheable)

Table 1: `effort` is deliberately not sent to Haiku 4.5 — the parameter errors on that model. The two model rows differ only in the ORDER of the prompt, not its content: the examples are retrieved per item and therefore differ on every request, so a prefix is cacheable only if the constant instructions come first.

2.3 Era-matched retrieval

Examples are drawn only from the train half, retrieved by content overlap with the item being extracted, and **filtered to the same era**. A PDF-era item is shown PDF-era examples. Cross-era retrieval would teach the wrong block anatomy, and until the modern gold set existed the configuration was maximally exposed to that — every available example was HTML-era. The retriever warns rather than silently falling back when an era has too few examples.

2.4 The prompt

Each model receives the schema-generated prompt, then the raw block. The prompt is not written by hand — it is built from `SCHEMA` by `extraction.common.build_prompt()`, so it cannot drift from the fields. It opens:

You are a strict JSON extractor. Given a block of raw meeting text, extract and return exactly one valid double-quoted JSON object with the keys below (any order). Only output the JSON—no commentary. Missing fields → empty string "" (or [] for list fields, 0 for counts).

followed by one line per field: name, the field’s `help` text, and for enums the permitted values. 3,089 characters, roughly 772 tokens, byte-identical on every request and therefore cacheable.

2.5 What the model is told about `request_type`

One field’s prompt line explains much of both methods’ behaviour and is worth quoting whole. This is exactly what the model sees:

```
- request_type: Type of request (often inferable from the case-number
suffix) --- one of: conditional_use, conditional_use_modification,
discretionary_review, variance, rezoning_map_amendment, planning_code_amendment,
general_plan_amendment, large_project_authorization, downtown_project,
ceqa_environmental, appeal_preliminary_negative_declaration, historic, coastal,
office_allocation, appeal, other, or ""
```

The parenthetical comes from the schema’s own help text. **It instructs the model to do what the labelling manual forbids** — the manual’s rule is that the suffix is a hint and the “Request for...” sentence decides. Section 4.2 measures who obeys it.

Because `PROMPT_INSTRUCTION` is generated from `help` strings, **every word of help text is prompt text**. That makes the help strings the cheapest lever in the project, and the one most easily overlooked.

2.6 The response schema

Responses are constrained by structured outputs (`output_config.format`) with a JSON schema generated from `SCHEMA`: lists become `array of string`, counts `integer`, enums a string with the vocabulary as `enum plus the empty string` — so “the block does not say” is a legal answer the model can select rather than something it must invent a way to express. All extracted fields are **required**; `additionalProperties` is false. **Every batch returned complete, parseable records with zero failures.**

This matters operationally. The labelling app’s older pre-fill path forces JSON with an assistant prefill (`{"role": "assistant", "content": "{"}`), which **returns a 400 on Sonnet 5, Opus 5 and the whole 4.6+ family**. It survives only because that path is pinned to Haiku 4.5. Structured outputs are the replacement and are strictly better: valid by construction rather than by hope.

2.7 The metric: why not exact match

The project’s original comparison, `field_match`, is exact string equality after case and whitespace folding. For enums that is right. For most of this schema it asks the wrong question, and the difference is not small: on identical predictions it scores Haiku at 82.3% where a field-aware comparison scores 91.2%.

What exact match was charging for:

- **A trailing full stop.** Gold `'do not take discretionary review and approve the project'`; prediction `'Do not take Discretionary Review and approve the project.'`
- **The minutes’ locational gloss.** Gold `'1233 Howard Street'`; prediction `'1233 Howard Street, east side between 8th and 9th Streets'`.
- **Labeller abbreviation.** On `project_descr` the gold is frequently a *prefix* of the prediction — the human stopped early. Requiring the extractor to reproduce an abbreviation measures nothing.
- **Punctuation inside an identifier.** `'motion no.: 14644'` versus `'Motion No. 14644'`; lot `'009, 010'` versus `'9,10'`.

`compare_field` chooses a rule by what the field *is*:

Field kind	Rule
enum, int	exact — approved and continued are different answers
date	parsed and compared as dates
identifier	punctuation and case dropped, digits preserved
permanent action id	instrument kept : a motion is not a resolution
names	set of surnames; order and honorifics carry nothing
address	the street address proper, ignoring the locational gloss
free text	containment, or token overlap ≥ 0.85

Table 2: Per-field comparison rules. `field_match` is retained, frozen, for results already scored with it.

Two deliberate refusals. **Motion versus resolution is not folded away** — they are different instruments, and the gold set turned out to be wrong about which on three items (§4.3). And **the**

address rule does not excuse the format: a separate predicate, `format_ok`, tracks whether `project_address` came back in the clean shape the geocoder and the APN join need. Loosening a correctness metric should not hide a real formatting requirement — that one is fixed in the prompt, not the ruler.

2.8 Two rates, never averaged

- **Accuracy** is measured *only where the gold record has a value*: when there is something to find, does the method find it?
- **Over-extraction** is measured *only where the gold record is blank*: does the method invent something when there is nothing there?

Scoring blanks as correct answers would put the regex extractor — which leaves most fields empty — at the top of the table for the worst possible reason.

3 Part II — Results

Table 3: Accuracy on the held-out half, by era. Gold version `g3`, 294 items (155 train / 139 test). Accuracy counts only field-values the gold record fills; over-extraction is the separate rate at which a method invents a value the gold leaves blank.

Method	Test (n=2100)	Test, HTML (n=1697)	Test, PDF (n=403)	Over-extraction (n=1931)
Deterministic rules	75.2%	74.0%	80.4%	1.7%
Haiku 4.5, few-shot	96.6%	97.2%	93.8%	4.5%
Haiku 4.5, few-shot, cached prefix	96.6%	97.2%	94.0%	4.2%

Table 4: Token cost, measured on the 294-item scored run and extrapolated to the 16,203 items in the corpus that carry a case number. Batch API pricing (half of list). The two rows differ only in the ORDER of the prompt: putting the constant instructions ahead of the per-item examples is what makes a prefix cacheable.

Configuration	Input	Cache read tokens per item	Cache write	Output	\$/1,000 items	\$/corpus (16,203 items)
Haiku 4.5, few-shot	10,541	0	0	564	\$6.68	\$108.25
Haiku 4.5, few-shot, cached prefix	4,980	5,353	208	568	\$4.31	\$69.81

3.1 The headline

Table 3 is the result. On the frozen test half, few-shot Haiku 4.5 scores **96.6%** against **75.2%** for the deterministic extractor — a gap of twenty-one points that no amount of regex maintenance is going to close.

Four findings, in order of how much they should change what you do:

Table 5: Field-level accuracy on the held-out half. Fields the schema marks **derived** are computed in Python and never asked of the model; fields with fewer than ten scored values are shown but should not be read as rates.

Section	Field	n	Rules	Haiku 4.5
<i>Identity</i>				
	case_number	139	97.8%	98.6%
	request_type	136	82.4%	97.1%
	action_instrument	50	94.0%	100.0%
	action_instrument_no	50	86.0%	100.0%
<i>Location</i>				
	project_address	119	89.1%	97.5%
	assessor_block	120	99.2%	100.0%
	lot_number	119	80.7%	100.0%
<i>Zoning & scale</i>				
	type_district	113	39.8%	96.5%
	type_district_descr	101	44.6%	92.1%
	height_and_bulk_district	98	85.7%	98.0%
	special_use_district	19	0.0%	84.2%
	project_descr	139	55.4%	99.3%
<i>Process</i>				
	staff_planner	137	76.6%	99.3%
	preliminary_recommendation	95	91.6%	100.0%
	preliminary_recommendation_category	95	83.2%	97.9%
	continued_to	49	95.9%	98.0%
	action	139	87.8%	95.0%
<i>Politics</i>				
	speakers	69	33.3%	69.6%
	ayes	122	83.6%	100.0%
	noes	17	5.9%	100.0%
	absent	52	76.9%	100.0%
	excused	1	100.0%	100.0%
<i>Conditions</i>				
	conditions_imposed	19	94.7%	100.0%
	project_modified	16	37.5%	75.0%
	modifications	17	0.0%	88.2%

Table 6: Where the deterministic rules are already enough: every field with at least 10 scored values on the held-out half where they reach 85%. ‘Verdict’ compares them with the best model — *rules win* where they score higher, *tied* within half a point, *adequate* otherwise.

Field	<i>n</i>	Rules	Best model	Verdict
assessor_block	120	99.2%	100.0%	rules adequate
case_number	139	97.8%	98.6%	rules adequate
continued_to	49	95.9%	98.0%	rules adequate
conditions_imposed	19	94.7%	100.0%	rules adequate
action_instrument	50	94.0%	100.0%	rules adequate
preliminary_recommendation	95	91.6%	100.0%	rules adequate
project_address	119	89.1%	97.5%	rules adequate
action	139	87.8%	95.0%	rules adequate
action_instrument_no	50	86.0%	100.0%	rules adequate
height_and_bulk_district	98	85.7%	98.0%	rules adequate

Table 7: Where a model is not optional: the same fields, below 85% on the rules. ‘Gap’ is the model’s lead in points.

Field	<i>n</i>	Rules	Best model	Gap
special_use_district	19	0.0%	84.2%	+84
modifications	17	0.0%	88.2%	+88
noes	17	5.9%	100.0%	+94
speakers	69	33.3%	69.6%	+36
project_modified	16	37.5%	75.0%	+38
type_district	113	39.8%	96.5%	+57
type_district_descr	101	44.6%	92.1%	+47
project_descr	139	55.4%	99.3%	+44
staff_planner	137	76.6%	99.3%	+23
absent	52	76.9%	100.0%	+23
lot_number	119	80.7%	100.0%	+19
request_type	136	82.4%	97.1%	+15
preliminary_recommendation_category	95	83.2%	97.9%	+15
ayes	122	83.6%	100.0%	+16

Table 8: Adjudicated disagreements. Every case where the model’s value differed from the hand label was judged against the source block. ‘Gold’ means the label stood; ‘model’ means the label was wrong and was corrected. The model column is the share of apparent model error that was never model error.

Verdict	Count	Share
The hand label was right	80	42.8%
The model was right; gold corrected	97	51.9%
Neither was right	10	5.3%
Total adjudicated	187	

Table 9: Evidence-span verification. On six fields the model must return the words in the block that support its value; the span is checked as a verbatim substring after whitespace normalisation. A failure is a triage signal, not a correction — the value is flagged, never nulled.

Configuration	Failures	Per 100 items
Haiku 4.5, few-shot	46	15.6
Haiku 4.5, few-shot, cached prefix	96	32.7

1. **The modern era works.** This is the first measurement of it. The model scores 97.2% on the HTML era and **94.0%** on the PDF era. A 3.2-point gap on a document format with different headers, different speaker notation and different case-number conventions is a good outcome, and it is the number that decides whether a corpus pass is worth running. It is also, notably, the regex’s *best* stratum (72.1%) — the modern minutes are more regular, which flatters rules and understates how much the model is adding.
2. **Prompt ordering is worth \$38 and costs nothing.** Placing the constant instructions ahead of the per-item examples makes the leading 3,600 tokens cacheable. The two orderings score *identically* on the test half (96.6% both) and within 0.2 points on every stratum, while measured input tokens fall from 10,541 to 4,980 per item (Table 4).
3. **Method matters far more than model.** In the first round three models spanned 2.9 points while the gap to the regex was 21.2. Nothing since has changed that: the work that moved the number was the prompt, the schema and the gold set, not the model.
4. **Over-extraction is the cost of the gain.** The model invents a value where the gold is blank on 4.2% of blank fields; the regex does so on 1.7%. This is the honest trade and it is why the two rates are reported separately and never averaged (§2.8). Evidence spans (Table 9) exist to make it detectable at extraction time rather than discoverable at analysis time.

3.2 Where the remaining error is

Table 5 gives the field-level breakdown. The residual error is concentrated, and it is concentrated in the places one would expect:

- **speakers** (69.6%) is the worst field by a wide margin and the least load-bearing. It is a list of people with a stance each, scored as a set, so one missing name fails the whole field. The stance rule was rewritten late (§5.4) and the field now records *how* a stance was determined, which makes the reliable subset separable.
- **project_modified** (75.0%, $n = 16$) and **special_use_district** (84.2%, $n = 19$) have too few scored values to read as rates.
- **type_district_descr** (92.1%) is almost entirely truncation — the model returning “Chinatown Residential Neighborhood Commercial” where the label has “... District”. The adjudication round judged twenty of these and the *model* won sixteen.
- Everything the parcel join depends on — **case_number**, **assessor_block**, **lot_number** — is at or above 98%.

3.3 Where the regex is still the right tool

Table 6 lists every field where the deterministic rules already reach 85% on the held-out half. Ten of the twenty-four scored fields clear that bar, and they are the ones with a fixed pattern in a fixed position: the case number, the parcel keys, the numbered instrument, the dates. On none of them does a model buy more than a few points, and on `case_number` the rules are within a point of the best model on 139 values.

`continued_to` is a cautionary tale. It scored **0%** in the first run — not because the extractor failed to find the date, but because it returned “April 21, 2005” where the schema stores ISO. Every value was right and every one counted as a miss. A date normalisation (and extending the pattern to the 3/12/98 form the archive also uses) took it to 95.9%, two points behind the best model on a field that had read as a total failure. The same lesson applied twice more since: `lot_number` and `assessor_block` were being scored before the storage layer ran on them, and `action_instrument` read 0% because the extractor was still writing the schema-v1 key that replaced it. None of the three was an extraction failure; all three were formatting compared across a difference neither side chose.

3.4 Where a model is not optional

Table 7 is the other half of the same cut. The pattern in it is the argument of this memo: the rules fail wherever the value is not in a fixed place. A special use district, a modification, a zoning district written out in words, the description itself — these are stated in prose that varies, and a regular expression written for one phrasing misses the next. The roll-call fields matter disproportionately, because they are the ones the politics of the dataset runs on.

4 Part III — How each method fails

The tables in this part are from the FIRST round: four methods, schema v1, gold version g1, and the 1998–2014 items only. They are kept because the failure taxonomy and the model comparison they support have not been re-run and are what motivated the configuration now recommended. Do not read their absolute rates as current — Part II has those, measured against gold g3 on both eras.

4.1 Failure classes

Classifying every miss over the full gold set, under the *exact* comparison, to show what the field-aware metric later absorbs:

Failure class	regex	haiku-4.5	sonnet-5	opus-5
Omitted — left blank	677	58	50	14
Different value	169	194	183	171
Substring — truncated or padded	140	175	202	199
Format only	75	115	123	124
Different values (list)	146	27	38	34
Partial — names missing or extra	36	6	5	3
Different date	3	2	2	2

Table 10: The two methods fail in categorically different ways.

Fifty-four percent of every regex failure is silence. It does not get the answer wrong; it declines to answer. That was a design choice — a blank is a question, a confident wrong value is a silent error — but at this volume the choice became the dominant cost.

The models almost never go silent, and Opus 5 hardly ever does: 14 blanks against 677. Their misses are “different value” and “substring” — the signature of a reader that always produces something, sometimes the wrong something and sometimes the right something trimmed differently. About a fifth of their apparent misses were format-only, which is what §2.7 addresses.

4.2 request_type: what the prompt is doing

Section 2.5 showed the prompt asserting that `request_type` is “often inferable from the case-number suffix”. Scoring every disagreement three ways — what the human said, what the method said, and what the *suffix alone* would give:

Method	Agrees with gold	Followed the suffix	Read the text	Neither
<code>regex</code>	82.8%	17.2%	0.0%	0.0%
<code>haiku-4.5</code>	93.0%	0.9%	4.0%	2.2%
<code>sonnet-5</code>	93.8%	1.3%	3.5%	1.3%
<code>opus-5</code>	94.7%	1.3%	3.1%	0.9%

Table 11: Full gold set, $n = 227$. “Followed the suffix” means the method’s answer equals the suffix-derived value where the human’s does not.

Every regex miss is “followed the suffix” — 100% of them, because that is the only mechanism it has: `derive_request_type()` is a lookup from case-number letters to enum values and cannot read the request. Its failures are exactly the cases the manual anticipates — 97.870C is a variance heard alongside a conditional use; 98.238ET is a Planning Code amendment, not a CEQA determination; four E cases are appeals of a preliminary negative declaration rather than generic environmental review.

The models almost never do this, *despite the prompt inviting it*. They read the description instead. This is the clearest single demonstration of what a model buys: the ability to follow a rule stated in prose rather than one encoded in a lookup table.

4.3 Some “model errors” are gold errors

Every disagreement between the model and the hand label that survived `compare_field` was queued and judged against the source block, with a three-way verdict: *gold right*, *model right*, *both wrong*. Table 8 is the result of that round.

More than half of the apparent model errors were label errors. That is the single most important number in this memo for interpreting every other number in it: an accuracy figure computed against an unadjudicated gold set is a lower bound of unknown size, and the bias is *larger* for the better models, because they are likelier to be right where the label is wrong.

The pattern was field-specific and diagnostic rather than random:

- `staff_planner` went 33–0 to the label — and that was a *prompt* bug, not a model failure. The help text said the answer was “initial plus surname” and gave only (D. WINSLOW: (415) 558-6335) as an example, so on the 1998–2001 minutes, which print a bare (WASHINGTON), the model found no initial and returned blank. Twenty-seven of the thirty-three were exactly

this. The help text now spells out both forms; the field scores 99.3% on the test half in this round.

- `type_district_descr` went 4–20 to the model, `project_address` 0–9, `request_type` 2–8. These are fields where the label was a paraphrase and the model copied the block.
- `lot_number` split 6–6 with 3 “both wrong”.

Two classes of correction were applied without a human verdict, because the source settles them: cases where the model returned nothing and the label’s value is printed verbatim in the block, and cases where the two values are identical after name normalisation (B.Wycko versus B. Wycko). Neither is a judgement call; both were logged.

4.4 The bugs the adjudication round exposed

Adjudication is usually described as a way to price gold error. In practice its larger return here was finding defects that no accuracy number would have revealed, because they were symmetric — present in gold *and* prediction:

- `lot_number` was declared `scalar` in the schema while the normaliser stored it as a list. `coerce_record` stringified `[8]` to `"[8]"`, which the next normalisation could not parse, and the value was dropped — silently, on whatever save happened to touch some other field. **122 of 232 gold records had lost their lot number.** They were recoverable only because the pre-migration snapshot is frozen and because the raw model replies are stored alongside the normalised ones.
- Three blocks held two agenda items each, so gold described item 1 while the model read item 2 and every field disagreed. The header regex had missed them because the archive prints `1a.2013.1521DDV` and `2004. 0164D` — no space where one was expected, a space where none was.
- Two blocks were truncated at a numbered line: a date (“...on December” + “4. The Project...”) and the enumerated modifications attached to an **ACTION:** line, which carried the vote away with them.

None of these would have been found by looking at an accuracy table.

4.5 Over-extraction, read carefully

The models fill a blank field on 5–9% of opportunities against 2.7% for regex. The number needs reading field by field, because “the human left it blank” is not “the answer is blank”.

The clearest case is `conditions_imposed`: every model scores **100%** accuracy where the gold has a value, and 23–33% over-extraction across the blanks. That field is filled on 11% of gold records while 32% of corpus blocks mention conditions somewhere — the gold is thin, so a model saying “yes” there is plausibly right.

Two alarming-looking figures are noise: `staff_planner` at 100% is on $n = 2$ blanks, `request_type` at 67% on $n = 3$. The over-extraction that looks like genuine model behaviour is `type_district/type_district_descr` (25–50%), where a model asked for a zoning district supplies one from the shape of the text even when the block does not name it.

5 Part IV — Improving the model

The tables in this part are from the FIRST round: four methods, schema v1, gold version g1, and the 1998–2014 items only. They are kept because the failure taxonomy and the model comparison they support have not been re-run and are what motivated the configuration now recommended. Do not read their absolute rates as current — Part II has those, measured against gold g3 on both eras.

5.1 The blank policy

If a model over-fills because nothing told it silence is a valid answer, saying so should help. v2 is identical to v1 — same model, items and response schema — except that it prepends:

BEFORE YOU EXTRACT --- the rules on blank fields:

1. A blank is a real answer, not a failure. It means "the block does not say".
2. Extract only what the block states. Do NOT infer a value from the case number, do NOT fill a field from general knowledge of San Francisco planning practice.
3. Before writing any non-empty value, check that you could point to the words in this block that support it.
4. Leaving five fields blank is better than guessing one.

Over-extraction falls from 9.2% to 7.3% and accuracy rises slightly (91.2% $\rightarrow$ 91.5%). The effect concentrates where it should — `type_district_descr` 50 $\rightarrow$ 33%, `conditions_imposed` 33 $\rightarrow$ 23%. It does not fix everything: the model still fills `type_district` on a third of blank items when told not to guess, which is disagreement about what the block says rather than disobedience.

5.2 Few-shot examples

Zero shots is the honest floor for a *method comparison* but not how the extractor should be run. Most of what the model still gets wrong is a *convention* rather than a reading failure — that `project_address` is the street address and not the cross-street gloss, that “Approved as proposed” on a discretionary-review case means the Commission declined to take DR. Conventions are what worked examples teach well and prose teaches badly.

The harness retrieves the k most similar labelled blocks **from the train half only**, by content overlap, and shows each with its gold record. Retrieval matters: a discretionary-review item is shown other discretionary-review items.

Configuration	Accuracy	Over-extraction	Whole records
haiku-4.5 (zero-shot)	91.2%	9.2%	29.8%
haiku-4.5-v2	91.5%	7.3%	31.6%
haiku-4.5-fs6	94.1%	5.6%	48.2%
haiku-4.5-fs6v2	93.8%	5.0%	49.1%
opus-5 (zero-shot)	93.5%	8.5%	40.4%

Table 12: Test half. Six retrieved examples buy more than a five-fold increase in model price, on every axis at once.

Six examples take Haiku past zero-shot Opus on accuracy, over-extraction and whole-record correctness simultaneously. Whole records — 29.8% to 48.2% — is the largest movement anywhere in this memo.

One cost caveat. Retrieved examples differ per request, so **they cannot be cached**: the 6-shot prefix is about 2,743 tokens per item and takes a corpus pass from \$18 to \$40. A *fixed* example set would be cacheable and much cheaper, at some accuracy cost. That comparison is untested and is the obvious next experiment.

5.3 What is not worth doing

Retrieval-augmented generation in its usual sense does not apply. The block is self-contained and about 450 tokens; the answer is entirely inside it. There is no external context to fetch. The only retrieval that helps is retrieval of *examples* (§5.2) — a different technique wearing the same name.

5.4 Speaker stance: what the model is asked to decide

speakers is the one field where the rule changed after the round reported here was scored, so its 69.6% should be read as measuring the *old* instruction.

Two decisions were taken. First, stance is anchored to **the request before the Commission**, not to the Commission’s eventual action. Anchoring to the action is circular: the same person saying the same words would be **support** when the Commission takes discretionary review and **oppose** when it does not, so the label would be determined by the outcome and could no longer be used to study outcomes. Under the rule adopted, on a discretionary-review item the request is to *take* DR, so the DR requestor is **support** even though they oppose the project.

Second, the field records *how* the stance was determined. The minutes carry an explicit vocabulary — (+), (−), (=) in 1,444 items and bare +/-/= line prefixes in 4,317 — but not on every item. The model is told to look for a marker first and set `stance_basis='marker'`, and only then to infer from what the speaker said and set `stance_basis='inferred'`. A stance read off a marker and a stance reasoned from remarks are different kinds of evidence; pooling them means the reliable half can never be isolated again. Of the stances in the current gold set, 574 are marker-derived and 34 inferred.

6 Part V — Recommendation

Run Haiku 4.5 with six retrieved, era-matched few-shot examples, with the instructions placed ahead of the examples so the prefix is cacheable. On the frozen test half it scores 96.6% overall, 97.2% on the HTML era and 94.0% on the PDF era, and one full pass over the 16,203 items carrying a case number costs **\$70** (Table 4).

The ordering deserves emphasis because it is free. The examples are retrieved per item and so differ on every request; with them in front, nothing in the prompt is cacheable. Moving the constant instruction block ahead of them turns 3,600 tokens per item into a cache read at a tenth the price — \$38 across the corpus — and the two orderings score identically.

6.1 What would change this recommendation

1. **A materially larger modern gold set.** 61 items across twelve years is five per year against a stratum of 4,855. The PDF-era figure of 94.0% rests on 403 scored field-values. It is enough

to justify a corpus pass; it is not enough to publish as a stratum-level accuracy without a confidence interval, and a second draw of 60 is the cheapest thing on this list.

2. **Evidence-verification failures at corpus scale.** 46 spans failed verification on 294 items — about 16 per 100 — and the rate should be watched on the full run rather than assumed stable. A rising rate is the earliest available signal that the model is inventing values.
3. **Nothing about model choice.** Sonnet 5 was dominated in the first round and Opus costs more than twice as much for a difference the prompt has since erased.

6.2 Other things worth doing, in order

1. **Draw a second 60 modern items.** The stratified draw is a one-command follow-up and it is the binding constraint on every era-level claim.
2. **Work the deferred typo queue.** Six disagreements are parked as punctuation and truncation differences. They do not change any conclusion; they do change the appendix’s accuracy figure by a few tenths, and they are the right kind of work to do once rather than continuously.
3. **Re-split the remaining merged blocks.** Today’s header fixes found 110 blocks holding a second agenda item and 104 truncated at a numbered line. Both classes are repaired in the stored blocks; the corpus parser should be re-run so the fixes hold from source.
4. **Watch speakers.** At 69.6% it is the weakest field, and the stance rule changed after this round was scored. It is also the least load-bearing field in the dataset.

7 Limitations

1. **The modern stratum is thin.** 61 items, five per year, standing in for 4,855. The PDF-era figure of 94.0% rests on 403 scored field-values and carries a confidence interval this memo does not compute. It is enough to decide whether to run the corpus; it is not a publishable stratum accuracy.
2. **The gold set still contains errors.** 187 disagreements were adjudicated and 52% of them were label errors, so the remaining rate is lower than it was — but it is not zero, and six known punctuation-level disagreements are deferred rather than resolved. Measured accuracy remains a lower bound, biased more for better models.
3. **The few-shot examples come from the train half of the same corpus.** Era-matched retrieval means a modern item is now shown modern examples, but there are only 36 modern examples to retrieve from, so the modern configuration is drawing on a much smaller pool than the HTML one. Part of the 3.4-point era gap may be this rather than the documents.
4. **One run per configuration.** No repeated sampling, so run-to-run variance is unmeasured; differences under about a point should not be read as real. The two prompt orderings differing by 0.0–0.2 points is consistent with them being the same configuration, which is the claim made for them, but it is not evidence that they are identical.
5. **Over-extraction is not cleanly attributable** where the gold is known to be under-filled, as on `conditions_imposed`.

6. **Rare fields are unmeasurable here.** `recused` has no gold values at all; `excused` has one. Their apparent scores are noise and the schema marks them `unmeasurable` so no table quotes them.
7. **The metric embeds judgement.** `compare_field` encodes decisions about what counts as the same fact. Each is documented in `extraction_common.py` and defensible, but a different reasonable analyst would draw one or two differently.
8. **This memo scores extraction, not the dataset.** A field can be extracted correctly from a block that was itself mis-split. Today's repairs found 110 merged blocks and 104 truncated ones; the accuracy figures assume the blocks are right, and that assumption is worth its own validation round.

8 Reproducing this

```
cd code/commission_minutes_processing
python gold_split.py                # freeze train/test (once)
python gold_split.py --verify       # has the gold changed since?
python bakeoff_extract.py --submit   # v1, all three models
python bakeoff_extract.py --submit --variant v2 --only haiku-4.5
python bakeoff_extract.py --submit --only haiku-4.5 --shots 6 --on test --tag fs6
python bakeoff_extract.py --collect  # poll, download
python bakeoff_report.py --test      # headline numbers
python bakeoff_report.py --test --exact # the old comparison, for reference
python bakeoff_report.py --request-type # the suffix-following analysis
python build_adjudication.py         # queue disagreements
python build_adjudication.py --tally  # what the verdicts say
python bakeoff_memo.py               # regenerate this memo's tables
```

Artefacts live in `code/commission_minutes_processing/bakeoff/`: `split.json` (the frozen split and its gold hash), `batches.json`, `raw_<method>.json`, `report.json` and `scores.csv`. This memo's tables are generated by `bakeoff_memo.py` into `bakeoff.tables.tex` and `\input` here, so no accuracy figure in this document is typed by hand.